Università degli Studi di Milano Bicocca

Clustering-based feature augmentation for fine-grained question classification on the TREC dataset

Presented by:
Giorgio Baselice, 886186
Hadis Forghani, 920040
Amaiza Jamil, 885861

0. Abstract..... 4
1. Introduction..... 4
2. The Dataset..... 5
3. Data Exploration..... 5
3.1 Dataset dimensions..... 5
3.2 Label structure..... 6
3.2.1 Coarse-grained labels (6 classes)..... 6
3.2.2 Fine-grained labels (50 classes)..... 6
3.3 Class distribution analysis..... 7
3.3.1 Coarse-grained Label Distribution..... 7
3.3.2 Fine-grained label distribution..... 8
3.4 Question length analysis..... 10
3.4.1 Overall length statistics..... 10
3.4.2 Question length by coarse category..... 11
3.5 Question word analysis..... 12
3.5.1 Question word frequency..... 12
3.5.2 Question word vs coarse label association..... 13
3.6 Vocabulary analysis..... 14
4. Text Representation..... 15
4.1. Text Preprocessing..... 15
4.1.1. TF-IDF Preprocessing..... 15
4.1.2. SBERT Preprocessing..... 16
4.2. TF-IDF Representation..... 16
4.3. SBERT Representation..... 16
4.4. Representation Conclusion..... 17
5. Clustering Techniques..... 17
5.1. Algorithm and Distance Metrics..... 17
5.2. Selection of the Number of Clusters..... 18
5.3. Clustering Results..... 18
5.4. Clustering Feature Extraction..... 19
6. Vector Construction..... 19
7. Question Classification..... 20
7.1. Evaluation Metrics..... 20
7.2. Results..... 20
8. Classification conclusions..... 21
8.1. Linear SVM VS Logistic Regression..... 21
8.2. SBERT VS TF-IDF..... 22
8.3. Clustering Information Results..... 22
8.4. All Data Combination Results..... 22
9. Interpretation of the Results..... 23

9.1. The paradox of low silhouette and high utility..... 23
9.2. Why linear SVM dominates..... 23
9.3. Complementarity of the Features..... 24
10. Error analysis..... 24
10.1 Overall error summary.....24
10.2 Coarse-level performance analysis.....24
10.2.1 Confusion Matrix (coarse-grained).....24
10.2.2 Performance metrics by coarse category.....25
10.3 Fine-grained performance analysis.....25
10.3.1 Classification report summary.....25
10.3.2 Confusion matrix (fine-grained).....26
10.4 Most confused class pairs.....27
10.5 Error analysis by question Word..... 27
10.6 Error distribution analysis.....28
10.9 Root cause analysis..... 28
11. Conclusions.....29
12. References.....30

0. Abstract

Question classification is a core task in Natural Language Processing, playing a crucial role in question answering systems. This study combines supervised and unsupervised learning approaches to improve question classification on the TREC Question Classification dataset without following the hierarchical clustering strategy. In particular, we employ soft clustering techniques to extract additional features, such as degrees of membership or distances from cluster centroids, which are then incorporated into a supervised classification model. Since the dataset has been extensively studied in the literature, no substantial performance gains were expected. However, experimental results show that clustering-derived features can provide complementary information to standard textual representations, leading to consistent improvements over baseline classifiers. These findings suggest that soft clustering can be a useful strategy for enriching feature representations in question classification tasks.

1. Introduction

Question classification is a fundamental component of Question Answering (QA) systems, as it serves as an initial layer of semantic understanding by assigning an input question to a predefined semantic category. The TREC Question Classification dataset has become a standard benchmark for this task due to its hierarchical label structure and the presence of concise, well-formed questions.

While supervised models achieve strong performance using lexical and semantic representations, this project explores whether unsupervised clustering techniques can be leveraged to extract additional semantic information and use it as auxiliary features to enhance fine-grained question classification.

The main objective of this project is to evaluate whether features derived from soft clustering (obtained by clustering questions using an unsupervised approach) can provide complementary information that improves classification performance on the TREC Question Classification dataset. To this end, we adopt a hybrid framework that combines unsupervised clustering with supervised classification.

In particular, the contributions of this work are the following:

- We propose a method for integrating soft clustering features into a supervised classification framework.
- We evaluate the impact of these features on a standard benchmark dataset.

The remainder of this report is organized as follows:

- Section 2 describes the dataset and the preprocessing steps.
- Section 3 presents the exploratory data analysis.
- Section 4 presents the text representation methods.
- Section 5 introduces the clustering techniques employed.

- Section 6 describes how the different input vectors for the classification models are built.
- Section 7 describes the supervised classification models and evaluates metrics.
- Section 8 discusses the experimental results.
- Section 9 provides interpretation of the results.
- Section 10 concludes the work.

2. The Dataset

The TREC Question Classification dataset, originally introduced by Li and Roth (2002) [1] , is a widely used benchmark for the task of question classification. It was originally introduced in the context of the TREC Question Answering tracks and consists of a collection of English questions annotated with semantic labels representing the expected answer type.

The dataset was downloaded from a publicly available online repository providing the standard train–test split using directly the code provided by that service [2].

The dataset contains 5,952 questions, divided into a training set of 5,452 instances and a test set of 500 instances. Questions are generally short and well-formed, with an average length of 10 tokens.

Each question is annotated using a hierarchical labelling scheme composed of six coarse-grained classes and 50 fine-grained subclasses. The coarse-grained classes represent broad semantic categories, while the fine-grained subclasses provide a more precise characterization of the expected answer type. In this work, we focus exclusively on the fine-grained classification task, and the hierarchical structure of the labels is not explicitly exploited.

The TREC dataset is particularly appropriate for this study due to its well-defined semantic categories and its widespread adoption in the literature, which enables comparison with previously reported results. Moreover, the semantic structure of the labels makes it a suitable benchmark for evaluating whether unsupervised clustering can capture latent semantic regularities useful for classification.

3. Data Exploration

Before applying clustering and classification methods, we perform an exploratory analysis of the TREC Question Classification dataset to better understand its characteristics and identify patterns that may inform feature engineering and model selection.

3.1 Dataset dimensions

The dataset contains a total of 5,952 questions, divided into a training set and a test set as follows:

Dataset	Number of Questions	Percentage

Training Set	5,400	91,6%
Test Set	500	8.4%
Total	**5,952**	**100%**

Table 3.1: Dataset split summary

3.2 Label structure

Each question is classified according to a two-level hierarchical scheme:

3.2.1 Coarse-grained labels (6 classes)

The **coarse-grained labels** represent high-level semantic categories:

Coarse-Label	Meaning
ABBR	Questions about abbreviations or acronymous
DESC	Questions asking for definitions, descriptions, or explanations
ENTY	Questions whose answer is an entity
HUM	Questions about people
LOC	Questions about locations
NUM	Questions expecting numerical answers

Table 3.2: Coarse-grained label descriptions with examples

3.2.2 Fine-grained labels (50 classes)

Each coarse class is further divided into **fine-grained categories**. The complete breakdown is as follows.

ABBR - Abbreviation	Meaning
abb	Expansion of an abbreviation
exp	Explanation of an abbreviation

DESC - Description	Meaning
Definition	Definition of a concept
description	General description
manner	Method or way something is done
reason	Explanation of why something happens

ENTY - Entity	Meaning
animal	Animals
body	Body parts
color	Colors
creative	Creative works (books, movies, songs)
currency	Monetary units
disease	Diseases or medical conditions
event	Events
food	Food items

instrument	Musical or technical instruments
language	Languages
letter	Letters or characters
other	Other entities
plant	Plants
product	Products
religion	Religions
sport	Sports
substance	Substances
symbol	Symbols
technique	Techniques
term	Technical terms
vehicle	Vehicles
word	Words

HUM - Human	Meaning
description	Description of a person
group	Group of people
individual	Specified person
title	Titles or roles

LOC - Location	Meaning
city	Cities
country	Countries
mountain	Mountains
other	Other locations
state	States or provinces

NUM - Numeric	Meaning
code	Codes or identifiers
count	Counts
date	Dates
distance	Distances
money	Monetary values
order	Ordering or ranking
other	Other numeric values
period	Time periods
percentage	Percentages
speed	Speeds
temperature	Temperatures
weight	Weights

3.3 Class distribution analysis

3.3.1 Coarse-grained Label Distribution

The distribution of coarse-grained labels reveals significant class imbalance:

Coarse label	Training Count	Training %	Test Count	Test %
ENTY	1,250	22.93%	94	18.8%
HUM	1,223	22.43%	65	13.0%
DESC	1,162	21.31%	138	27.6%
NUM	896	16.43%	113	22.6%
LOC	835	15.32%	81	16.2%
ABBR	86	1.58%	9	1.8%

Table 3.3: Coarse-grained label distribution

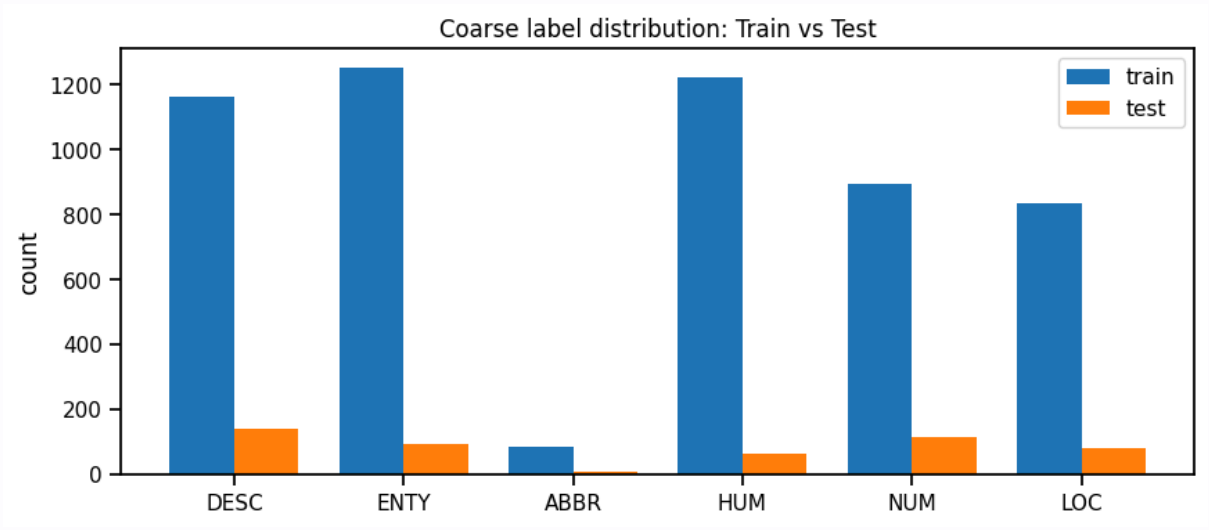

Figure 3.1: Coarse label distribution comparing training and test sets

Key observations:

1. **Severe underrepresentation of ABBR:** the ABBR (abbreviation) class contains only 86 training instances (1.58%), making it significantly underrepresented compared to other categories. This poses a challenge for classification, as the model has limited examples to learn from.
2. **Relatively balanced major classes**: the four largest classes (ENTY, HUM, DESC, NUM) are relatively balanced, each containing between 16% and 23% of the training data.
3. **Train-test distribution shift:** the set shows a slightly different distribution, with DESC being more prominent (27.6% vs 21.31%) and HUM being less frequent (13.0% vs 22.43%). This distribution shift may affect model performance on the test set.

3.3.2 Fine-grained label distribution

The fine-grained label distribution shows even more pronounced imbalance:

Top 10 Most Frequent Fine-Grained Classes (Training Set):

Fine label	Coarse	Count	Percentage
ind	HUM	962	17.64%
other	ENTY	733	13.44%
def	DESC	421	7.72%
count	NUM	363	6.66%
desc	DESC	321	5.89%
manner	DESC	276	5.06%
date	NUM	218	4.00%
reason	DESC	207	3.80%
cremat	ENTY	191	3.50%
gr	HUM	189	3.47%

Table 3.4: Top 10 most frequent fine-grained classes

Bottom 10 Least Frequent Fine-Grained Classes (Training Set):

Fine label	Coarse	Count	Percentage
currency	ENTY	11	0.20%
body	ENTY	11	0.20%
speed	NUM	10	0.18%
letter	ENTY	9	0.17%
religion	ENTY	9	0.17%
symbol	ENTY	9	0.17%
temp	NUM	8	0.15%
code	NUM	6	0.11%
exp	ABBR	4	0.07%
ord	NUM	4	0.07%

Table 3.4: Bottom 10 least frequent fine-grained classes

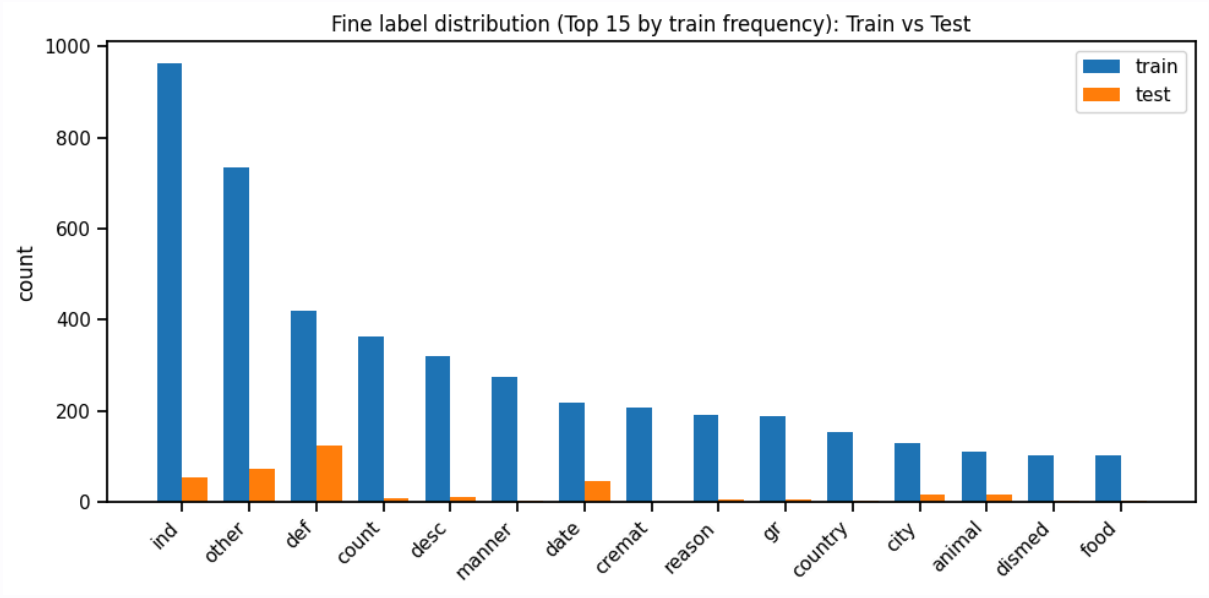

Figure 3.2: fine-grained label distribution (top 15 classes by training frequency) comparing training and test sets.

Key observations:

1. **Extreme class imbalance:** the imbalance ratio between the most frequent class (ind: 962 samples) and the least frequent class (ord: 4 samples) is 240.5:1. This severe imbalance poses significant challenges for classification.
2. **Long-Tail Distribution:** the distribution follows a long-tail pattern where a few classes dominate while many classes have very few instances. Approximately 20 classes (43%) have fewer than 50 training instances each.
3. **HUM:ind dominance:** The HUM:ind (individual person) class alone accounts for 17.64% of all training data, making it the most frequent class by a substantial margin.
4. **ENTY fragmentation**: the ENTY coarse category is divided into many fine-grained subclasses (22 subclasses), contributing to within-category confusion. Many ENTY subclasses have very few training instances.
5. **Implications for evaluation**: Given this severe imbalance, F1-Macro is essential for evaluating model performance, as accuracy alone would be misleading (a model predicting only the top 5 classes could achieve high accuracy while failing on minority classes).

3.4 Question length analysis

We analyze the length of questions in terms of number of tokens (words) to understand the textual characteristics of the dataset.

3.4.1 Overall length statistics

Statistic	Training Set	Test Set

Mean Length	10.20 tokens	7.52 tokens
Median Length	10 tokens	7 tokens
Standard Deviation	3.89 tokens	2.73 tokens
Minimum Length	3 tokens	4 tokens
Maximum Length	37 tokens	17 tokens
25th percentile	7 tokens	5 tokens
75th percentile	12 tokens	9 tokens

Table 3.5: Question length statistics (in tokens)

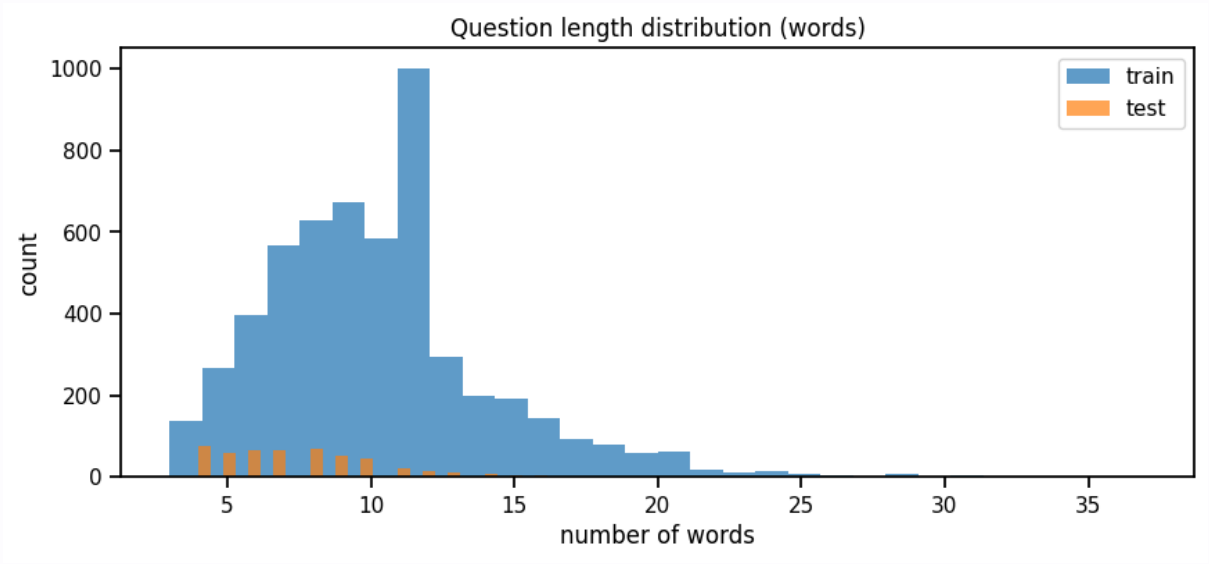

Figure 3.3: Distribution of question lengths (in words) for training and test sets.

Key observations:

1. **Short, concise questions:** questions are predominantly short, with a mean length of approximately 10 tokens in the training set. The distribution is right-skewed, with most questions containing between 7 and 12 tokens.
2. **Train-test length difference:** interestingly, the test set contains shorter questions on average (7.52 vs 10.20 tokens). This difference may reflect a deliberate choice in test set construction or natural variation in question complexity.
3. **Outliers present:** a small number of questions are substantially longer (up to 37 tokens in training), though 75% of questions contain 12 or fewer tokens.
4. **Implications for representation:** the compact nature of questions makes them suitable for both bag-of-words (TF-IDF) representations and dense embedding (SBERT) approaches. However, the brevity also means that each word carries significant discriminative weight.

3.4.2 Question length by coarse category

Coarse label	Mean length	Std Dev	Min	Max
DESC	8.81	3.84	3	37
ENTY	10.77	3.71	5	31

ABBR	8.01	3.27	4	18
HUM	11.16	4.42	3	34
NUM	10.35	3.35	5	29
LOC	9.97	3.29	4	29

Table 3.6: Question length statistics by coarse category

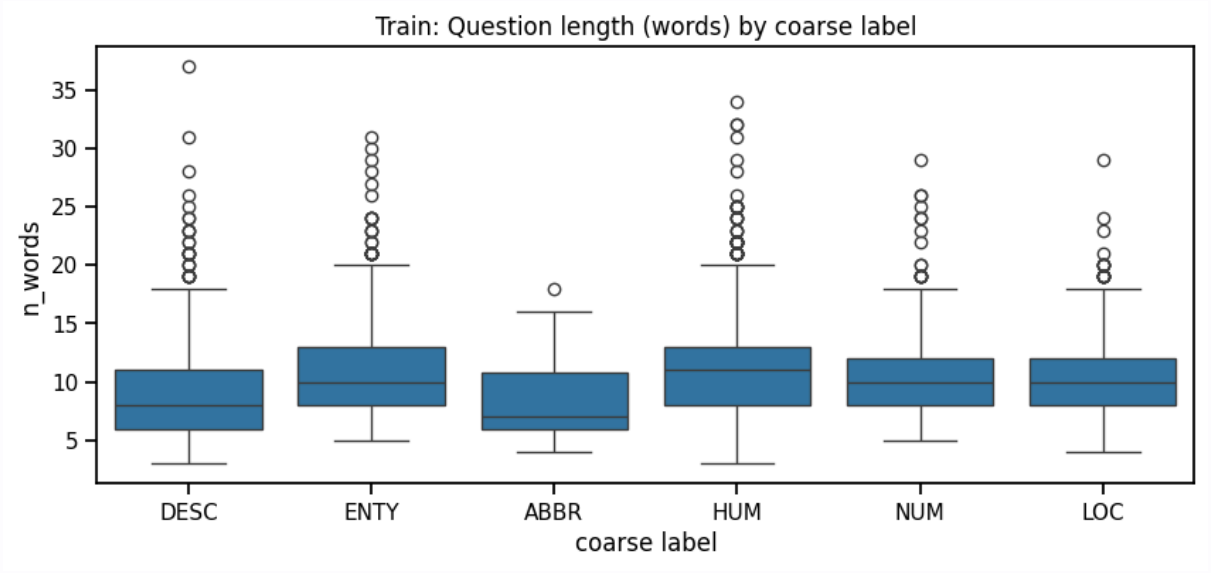

Figure 3.4: Box plot showing question length distribution by coarse category (training set).

Key observations:

1. **ABBR and DESC are shortest:** questions about abbreviations (ABBR: mean 8.01) and descriptions (DESC: mean 8.81) tend to be the shortest. This makes sense as abbreviation questions are inherently concise (e.g., "What does NASA stand for?").
2. **HUM questions are longest:** questions about people (HUM: mean 11.16) tend to be the longest, likely because they often include additional context about the person being asked about.
3. **Similar variability:** all categories show similar standard deviations (3.27-4.42), indicating consistent variability in question length across categories.
4. **Outliers in DESC and HUM:** the box plot reveals that DESC and HUM categories contain the most extreme outliers, with some questions reaching 34-37 tokens.

3.5 Question word analysis

We analyze the distribution of question words (wh-words) to understand the syntactic patterns in the dataset and their relationship to question categories.

3.5.1 Question word frequency

Question word	Count	Percentage
What	3,242	59.46%
How	764	14.01%
Who	559	10.25%

Where	273	5.01%
Other	167	3.06%
When	131	2.40%
Which	105	1.93%
Why	103	1.89%
Name	90	1.65%
Whose	14	0.26%
Whom	4	0.07%

Table 3.7: Question word frequency distribution (training set).

Key observations:

1. **"What" dominates:** nearly 60% of all questions begin with "What", making it the most common question word by a substantial margin.
2. **"How" and "Who" are secondary:** together with "What", these three question words account for approximately 84% of all questions.
3. **Rare question words:** "Whose" and "Whom" are extremely rare (0.26% and 0.07% respectively), which may make questions starting with these words difficult to classify correctly.
4. **"Name" as imperative:** the presence of "Name" (1.65%) indicates that not all instances are strictly interrogative sentences; some use imperative constructions (e.g., "Name the largest planet").

3.5.2 Question word vs coarse label association

The cross-tabulation reveals strong associations between question words and coarse categories:

Question word	DESC	ENTY	ABBR	HUM	NUM	LOC
What	23.1%	34.3%	2.5%	16.4%	7.5%	16.1%
How	36.4%	0.9%	0.0%	0.0%	62.7%	0.0%
Who	0.0%	0.4%	0.0%	99.6%	0.0%	0.0%
Where	5.1%	0.7%	0.0%	0.0%	0.7%	93.4%
When	0.0%	0.8%	0.8%	3.1%	94.7%	0.8%
Which	0.0%	41.0%	0.0%	43.8%	0.0%	15.2%
Why	100.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Name	0.0%	37.8%	0.0%	54.4%	0.0%	7.8%
Whom	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%
Whose	0.0%	0.0%	0.0%	100.0%	0.0%	0.0%

Table 3.8: Percentage distribution of coarse labels for each question word (rows sum to 100%).

Key observations:

1. **Strong discriminative signals:**
   "Who/Whom/Whose" → HUM (99-100%): questions beginning with these words

almost exclusively belong to HUM category
"Where" → LOC (93.4%): location questions are strongly indicated by "Where".
"When" → NUM (94.7%): temporal questions mostly expect numeric answers.
"Why" → DESC (100%): all "Why" questions are classified as DESC (seeking explanations/reasons).

2. **Ambiguous question words:**
   "What" is distributed across all categories, making it the least discriminative question word (though it still favors ENTY at 34.3%).
   "How" is split between DESC (36.4%) for manner questions and NUM (62.7%) for quantity questions ("How many", "How much").
   "Which" is ambiguous between ENTY (41.0%) and HUM (43.8%).
3. **Implications for preprocessing:** These strong associations justify our decision to retain question words in the TF-IDF representation (no stop-word removal). Removing question words would eliminate highly discriminative features.

3.6 Vocabulary analysis

Metric	Value
Total tokens	55,636
Unique words	8,677
Hapax Legomena (words appearing once)	5,199
Average unique words per question	9.86
Type-Token ratio	0.156

Table 3.9: Vocabulary statistics (training set).

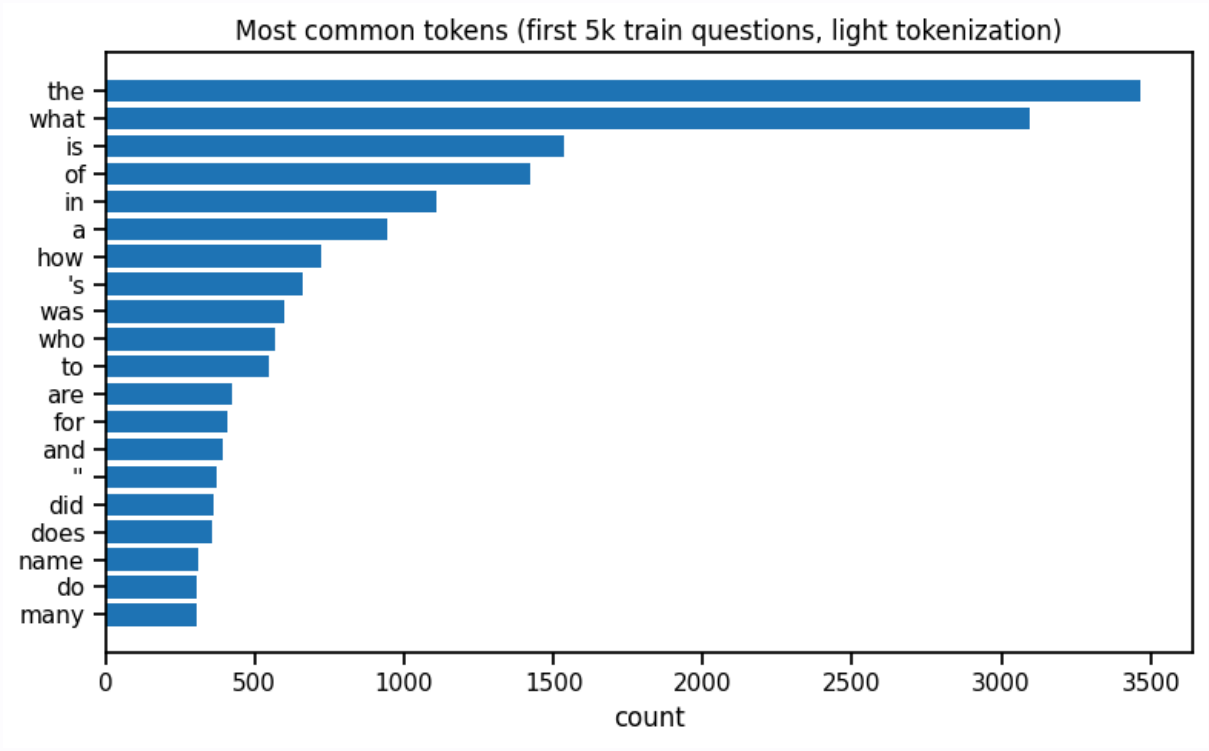

Figure 3.5: Top 20 most frequent tokens in the training set.

Key observations:

1. **High hapax lagomena rate:** nearly 60% of unique words appear only once in the corpus. This indicates a diverse vocabulary with many rare terms, justifying our use of max_features=10000 in TF-IDF to filter noise while retaining discriminative terms.
2. **Function words dominate:** the most frequent tokens are primarily function words ("the", "is", "of", "in", "a") and question words ("what", "how", "who"). This is typical for English text.
3. **Question-specific patterns:** tokens like "did", "does", "do" (auxiliary verbs in questions), "name" (imperative), and "many" (quantity questions) reflect the interrogative nature of the dataset.
4. **Implications for TF-IDF:** while function words are frequent, TF-IDF's IDF component will naturally down-weight them. The presence of discriminative question words ("what", "how", "who") among the most frequent tokens confirms their importance for classification.

4. Text Representation

Given that TREC questions are short, yet grammatically and semantically dense, it is crucial to adopt representations that capture both surface-level patterns and deeper semantic meaning. In this project, we combine two complementary approaches to textual representation: TF-IDF vectors and contextual embeddings obtained from a pre-trained Sentence-BERT (SBERT) model. The combination leverages both syntactic information, captured by TF-IDF, and semantic information, encoded by SBERT embeddings.

4.1. Text Preprocessing

Text preprocessing is a fundamental step in all NLP projects. This task consists of cleaning data to transform it in a more suitable form for further transformations. Since we use two

distinct representation methods, preprocessing is tailored to each approach.

- SBERT embeddings do not require extensive preprocessing, as the pre-trained model is designed to handle raw natural sentences. SBERT is sensitive to word order, syntax, and function words, and is robust to noise such as punctuation and casing.
- TF-IDF, on the other hand, are highly sensitive to tokenization, casing, and noise, so more careful preprocessing is required.

For both the representation models we apply normalization of the whitespaces and the stripping of leading and trailing spacing.

4.1.1. TF-IDF Preprocessing

For TF-IDF, preprocessing involves the following steps:

1. **Lowercasing** – to reduce the vocabulary size by treating tokens with different cases as identical.
2. **Tokenization with SpaCy** – to split text into meaningful units (tokens) while handling punctuation and special characters.
3. **Retention of unigrams and bigrams** – to capture both single-word features and frequent two-word patterns, which are often discriminative in short questions.

This preprocessing ensures that the TF-IDF vectors accurately reflect the lexical content of the questions while maintaining relevant multi-word expressions.

4.1.2. SBERT Preprocessing

No further preprocessing is applied to the input text for SBERT, as the model has been pre-trained on natural, raw sentences and can handle punctuation, casing, and function words effectively.

4.2. TF-IDF Representation

The TF-IDF representation is generated using the TfidfVectorizer function from the **scikit-learn** package [3]. The following settings are applied:

- **Custom tokenizer:** we use the SpaCy tokenizer defined in the preprocessing step to ensure consistent tokenization.
- **No additional preprocessing within the vectorizer:** since preprocessing is already applied separately.
- **L2 normalization:** to scale vectors to unit norm, ensuring comparability and stabilizing distance-based models or classifiers.
- **Maximum document frequency (max_df=0.95):** to remove very common terms that are unlikely to be discriminative, e.g., stopwords that appear in nearly all questions.
- **Maximum number of features (max_features=10000):** to limit dimensionality and computational cost while retaining the most informative tokens.
- **Retention of unigrams and bigrams:** to capture important word combinations.

These choices balance **informativeness**, **robustness**, and **computational efficiency**, producing a sparse, high-dimensional representation suitable for supervised classification.

4.3. SBERT Representation

The SBERT embeddings are generated using the SentenceTransformer class from the **sentence-transformers** package [4]. Specifically, we use the pre-trained model "all-mpnet-base-v2", which produces **dense, context-aware sentence embeddings**.

Property	Value
Model	all-mpnet-base-v2
Embedding Dimension	768
Training Data	1+ billion sentence pairs
Pooling	Mean polling

Table 4.1: SBERT model specifications

SBERT captures semantic relationships between words and phrases, making it particularly suitable for short, information-dense questions. Since TREC questions are often concise and grammatically rich, SBERT embeddings provide robust semantic features that complement the syntactic information captured by TF-IDF.

The resulting embeddings are **fixed-length dense vectors** (768-dimensional vectors) which are then combined with TF-IDF vectors and features derived from soft clustering to form the final input representation for the supervised classification models.

4.4. Representation Conclusion

TF-IDF captures the syntactic and lexical structure of the questions by quantifying the importance of terms in the dataset, highlighting discriminative words that can distinguish between classes. In contrast, SBERT embeddings provide dense, context-aware representations that encode semantic information at the sentence level, allowing the model to capture relationships and meaning beyond individual word occurrences.

By combining TF-IDF and SBERT features, we aim to leverage the strengths of both approaches: the interpretability and sparsity of TF-IDF, along with the rich semantic encoding of SBERT. These representations are subsequently augmented with features derived from soft clustering, forming the final input vectors for the supervised classification models.

5. Clustering Techniques

Clustering is employed in this project as an **intermediate, unsupervised step**, rather than as a final task. Its primary purposes are to:

- Capture latent semantic groupings of questions.
- Generate informative, high-level features.
- Improve downstream classification performance.

Consequently, the clustering algorithm should be evaluated primarily in terms of stability, semantic coherence, and usefulness of the derived features, rather than purely intrinsic clustering metrics.

Although clustering is unsupervised, it is crucial to avoid **data leakage**. Cluster centroids are learned exclusively on the **training set**, and test instances are subsequently assigned to the

nearest cluster centroid. This ensures that no information from the test set influences the construction of clustering-derived features.

5.1. Algorithm and Distance Metrics

The main algorithm used is **K-Means**, chosen because:

- It scales efficiently to the size of the dataset.
- It works well with dense, continuous embeddings such as SBERT vectors.
- It produces centroids, which are ideal for constructing **distance-based** and **similarity-based features** for the supervised classifier.
- It can use cosine similarity to evaluate the clustering.

5.2. Selection of the Number of Clusters

To determine the optimal number of clusters K, we rely solely on the **Silhouette Score**, an intrinsic metric that measures how well each point lies within its assigned cluster relative to other clusters. Higher silhouette values indicate more coherent and well-separated clusters.

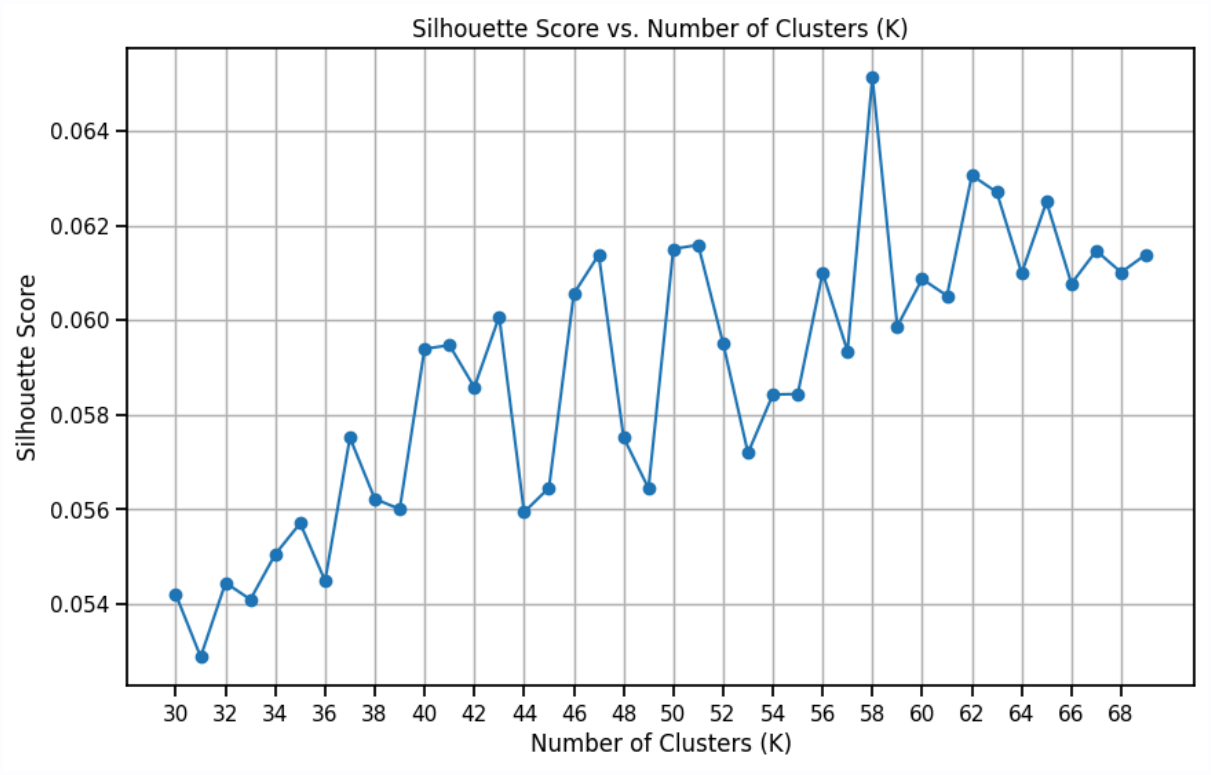

Figure 5.1: Silhouette score vs number of clusters

The optimal K=58 was selected based on the highest silhouette score achieved.

5.3. Clustering Results

Metric	Value
Optimal number of clusters (K)	58
Silhouette score	0.0651
Cluster purity (coarse labels)	0.6110 (61.1%)
Cluster purity (fine labels)	0.4485 (44.9%)

Table 5.1: Clustering evaluation metrics.

Experiments were conducted to evaluate different values of K, ranging from 30 to 70. Silhouette scores ranged from 0.0520 to 0.0651, with the best score obtained with K=58. For this configuration, the corresponding cluster purity with respect to coarse-grained labels was 0.6110, while the cluster purity with respect to fine-grained labels was 0.4485.

The silhouette score of 0.0651 is relatively modest, which is expected for several reasons:

1. **High-dimensional embedding space** (768 dimensions) where distances are naturally more uniform.
2. **Semantic overlap** between question categories in embedding space.
3. **Short text variability** within the same semantic category.

However, as we will demonstrate in Section 8, even clusters with moderate coherence provide **meaningful auxiliary information** for downstream classification. The cluster purity of 61.1% (coarse) indicates that the clusters capture semantically meaningful groupings that correlate with the annotation scheme, while 44.9% purity (fine) reflects the inherent difficulty of fine-grained distinctions.

The goal of clustering is not to recover the original question labels, but to generate high-level semantic features that complement TF-IDF and SBERT representations.

5.4. Clustering Feature Extraction

From each cluster, features are extracted for each question in both the training and test set:

Feature type	Dimensions	Description
Distance to all centroids	58	Euclidean distance to each cluster center
Cosine similarity to all centroids	58	Similarity score to each cluster center
Distance to assigned centroid	1	Distance to the nearest cluster
Total	**117**	

Table 5.2: Clustering feature composition

These soft assignment features capture:

- **semantic positioning** relative to learned prototypes.

- **boundary information** for questions near cluster edges.
- **degree of membership** to multiple semantic groups.

6. Vector Construction

Given all the components constructed in the previous step it is possible to build all the different vectors that are used for the different test of the question classification task:

Vector name	Components	Train dimensions	Test dimensions
train_tfidf / test_tfidf	TF-IDF only	(5,452 x 10,000)	(500 x 10,000)
train_sbert / test_sbert	SBERT only	(5,452 x 768)	(500 x 768)
train_tfidf_feat / test_tfidf_feat	TF-IDF + Clustering	(5,452 x 10,117)	(500 x 10,117)
train_sbert_feat / test_sbert_feat	SBERT + clustering	(5,452 x 885)	(500 x 885)
Train_full / test_full	TF-IDF + SBERT + Clustering	(5,452 x 10,885)	(500 x 10,885)

Table 6.1: Feature vector configurations.

For optimal performance, each feature group (TF-IDF, SBERT, Clustering) should be normalized separately before concatenation to ensure comparable scales. In the current implementation, L2 normalization is applied to TF-IDF vectors, while SBERT embeddings are used as-is (already normalized by the model).

7. Question Classification

The final task in this project is **fine-grained question classification** on the TREC dataset. The goal is to assign each question to one of the 50 fine-grained classes using features derived from textual representations and clustering.

Two linear, widely used supervised classifiers were evaluated:

1. **Logistic Regression** - a probabilistic linear model suitable for multi-class classification.
2. **Linear Support Vector Machine (SVM)** - a margin-based linear classifier.

Both classifiers were trained and tested using the different combination of information built previously.

7.1. Evaluation Metrics

Performance was measured using standard multi-class classification metrics:

- **Accuracy** – proportion of correctly classified questions.
- **Macro F1-score** – averaged F1-score across all classes, treating each class equally, to account for class imbalance.

- **Micro F1-score** – computes F1 globally by counting total true positives, false negatives, and false positives, emphasizing performance on frequent classes.
- **Weighted F1-score** – averages F1 per class weighted by class support, balancing contributions of both frequent and rare classes.
- **Confusion analysis** – to inspect which classes benefit most from clustering-derived features.

This setup allows a thorough evaluation of whether the addition of clustering features improves classification and whether semantic and syntactic representations are complementary.

7.2. Results

We summarize the results of the clustering task in the following table

Configuration	Classifier	Accuracy	F1-Micro	F1-Macro	F1-Weighted
Only TF-IDF	Logistic Regression	0.676	0.676	0.287	0.605
Only TF-IDF	Linear SVM	0.844	0.844	0.739	0.833
Only SBERT	Logistic Regression	0.720	0.720	0.347	0.675
Only SBERT	Linear SVM	0.870	0.870	0.723	0.868
TF-IDF + Cluster	Logistic Regression	0.786	0.786	0.476	0.747
TF-IDF + Cluster	Linear SVM	0.892	0.892	0.782	0.890
SBERT + Cluster	Logistic Regression	0.756	0.756	0.429	0.722
SBERT + Cluster	Linear SVM	0.864	0.864	0.725	0.862
All Features	Logistic Regression	0.814	0.814	0.546	0.782
All Features	Linear SVM	0.922	0.922	0.866	0.919

Table 7.1: Classification results across all configurations.

8. Classification conclusions

In this section are presented the different results we have obtained in the classification task.

8.1. Linear SVM VS Logistic Regression

Starting the analysis by observing the most striking difference in the results obtained with the classification, i.e. the performance gap between Linear SVM and Logistic Regression

Measure	Only TF-IDF Difference (SVM-LR)	Only SBERT Difference (SVM-LR)
Accuracy	0.168	0.150
F1-Macro	0.452	0.376

F1-Weighted	0.228	0.193

Measure	TF-IDF + Clustering Features Difference (SVM-LR)	SBERT + Clustering Features Difference (SVM-LR)
Accuracy	0.106	0.108
F1-Macro	0.306	0.296
F1-Weighted	0.143	0.140

Measure	All Features Difference (SVM-LR)
Accuracy	0.108
F1-Macro	0.320
F1-Weighted	0.137

Table 8.1: Performance comparison between Linear SVM and Logistic Regression

Linear SVM consistently outperforms Logistic Regression in all the metrics. This suggests that the maximum-margin approach of SVM is better suited for the high-dimensional, multi-class nature of fine-grained question classification.

8.2. SBERT VS TF-IDF

The different representations, TF-IDF and SBERT, for the text, even if combined in the result, showed different results.
If we consider accuracy and F1-Micro:

- SBERT outperforms TF-IDF with both the Logistic Regression Model and the Linear SVM if we do not add any clustering information.
- By adding the clustering feature, TF-IDF shows the most improvement, exceeding the results obtained by SBERT

The same can be observed for all the other metrics

8.3. Clustering Information Results

Adding the clustering information to the text representations proved to lead to improvements in the scores of the different metrics. In particular, the most observable changes can be observed in the results of the Logistic Regression Classifier.

The greater improvement can be recorded with the TF-IDF representation.

TF-IDF Measure	Logistic Regression Difference (with – without)	Linear SVM Difference (with – without)
Accuracy	0.110	0.048
F1-Macro	0.189	0.043
F1-Weighted	0.142	0.057

Table 8.2: Impact of clustering on TF-IDF representation.

In the case of the SBERT representation, the improvements can be mainly observed with the Logistic Regression model, while with the Linear SVM the results are almost identical.

SBERT Measure	Logistic Regression Difference (with – without)	Linear SVM Difference (with – without)
Accuracy	0.036	-0.006
F1-Macro	0.082	0.002
F1-Weighted	0.047	- 0.006

Table 8.3: Impact of clustering on SBERT representation.

8.4. All Data Combination Results

Adding all the features together proved to be the best solution according to all the metrics used.

Logistic Regression Measure	Difference (All data – Other Best Result)
Accuracy	0.028
F1-Macro	0.070
F1-Weighted	0.035

Linear SVM Measure	Difference (All data – Other Best Result)
Accuracy	0.030
F1-Macro	0.084
F1-Weighted	0.029

Table 8.4: Performance improvement with combined features.

These results confirm that **TF-IDF, SBERT, and clustering features provide complementary information**. TF-IDF captures surface-level lexical and syntactic patterns, SBERT encodes contextual semantic meaning, while clustering-derived features introduce high-level latent semantic structure. When combined, these representations enable the classifiers to leverage both low-level and abstract information, leading to consistent and robust performance gains.

Overall, the observed improvements across multiple metrics and classifiers demonstrate that **soft clustering features constitute a valuable auxiliary signal** for fine-grained question classification, even when the clustering itself exhibits moderate intrinsic quality.

9. Interpretation of the Results

9.1. The paradox of low silhouette and high utility

An important finding of this project is that clustering quality metrics do not directly predict downstream classification utility. Despite achieving a silhouette score of only 0.0663, the cluster-derived features improved TF-IDF + SVM accuracy by 5.0% and TF-IDF + logistic regression accuracy by 9.0%.

We can conclude that the clustering doesn't need to produce well-separated clusters to be useful. Instead, the distance and similarity features provide

- Soft semantic coordinates: positioning each question relative to 62 semantic prototypes.
- Boundary information: questions near cluster boundaries receive similar distances to multiple centroids, encoding uncertainty.
- Dimensionality reduction: compressing 768-dimensional SBERT space into 125 interpretable features.

9.2. Why linear SVM dominates

The consistent superiority of linear SVM over logistic regression can be attributed to several factors:

1. Maximum margin principle: SVM finds the hyperplane that maximizes the margin between classes, which is particularly effective when classes have subtle distinctions (as fine-grained question classification).
2. High-dimensional effectiveness: linear SVM handles high-dimensional feature space efficiently without overfitting, which is crucial given our feature dimensionality.
3. Robustness to class imbalance: some fine-grained classes have very few examples. SVM handles this better due to its focus on support vectors rather than all training instances.
4. Sparse feature handling: SVM with L2 regularization effectively manages the sparse TF-IDF features while maintaining discriminative power.

9.3. Complementarity of the Features

The superior performance of the full feature set demonstrated the TF-IDF, SBERT, and clustering feature capture complementary information.

Feature Type	Information Captured
TF-IDF	Lexical patterns, question words (what, who, how), domain-specific terms
SBERT	Semantic similarity, contextual meaning
Clustering Features	Latent groupings, soft category membership, prototype distances

Table 9.1: Complementary information captured by each feature type.

10. Error analysis

To better understand model behavior, we analyze the confusion matrix and error patterns of the best-performing model (All Feature + Linear SVM) on the test set.

10.1 Overall error summary

Metric	Value

Total test samples	500
Correctly classified	461 (92.20%)
Misclassified	39 (7.80%)

Table 10.1: Overall classification performance summary

The model achieves strong overall performance with 92.20% accuracy, misclassifying only 39 out of 500 test questions. However, as we will see, these errors are not uniformly distributed across classes.

10.2 Coarse-level performance analysis

10.2.1 Confusion Matrix (coarse-grained)

	Pred:DESC	Pred:ENTY	Pred:ABBR	Pred:HUM	Pred:NUM	Pred:LOC
True:DESC	135	1	2	0	0	0
True:ENTY	5	84	0	4	1	0
True:ABBR	1	0	8	0	0	0
True:HUM	2	2	0	61	0	0
True:NUM	1	10	0	1	101	0
True:LOC	1	54	0	0	0	26

Table 10.2: Coarse-level confusion matrix (rows = true labels, columns = prediction labels).

10.2.2 Performance metrics by coarse category

Coarse category	Precision	Recall	F1-Score	Support	Error Rate
DESC	0.931	0.978	0.954	138	2.90%
ENTY	0.556	0.894	0.686	94	17.02%
ABBR	0.800	0.889	0.842	9	11.11%
HUM	0.924	0.938	0.931	65	6.15%
NUM	0.990	0.894	0.940	113	7.96%
LOC	1.000	0.321	0.486	81	6.17%

Table 10.3: Classification performance by coarse category.

Key observations:

1. **DESC performs best:** the DESC (description) category achieves the highest F1-score (0.954) with excellent precision (0.931) and recall (0.978). Only 4 out of 138 DESC questions are misclassified.
2. **LOC has severe recall problems:** despite perfect precision (1.000), the LOC (location) category suffers from extremely low recall (0.321). This means the model correctly identifies LOC questions when it predicts them, but it fails to recognize most LOC questions, predicting them as other categories instead.
3. **LOC → ENTY confusion:** the confusion matrix reveals that 54 out of 81 LOC questions (66.7%) are misclassified as ENTY. This is the dominant error pattern at the coarse level.

4. **ENTY has low precision:** the ENTY category has low precision (0.556) because many questions from other categories (especially LOC) are incorrectly predicted as ENTY.

10.3 Fine-grained performance analysis

10.3.1 Classification report summary

The fine-grained classification report reveals heterogeneous performance across the 39 classes present in the test set:

Class	Precision	Recall	F1-Score	Support
ind	0.95	1.00	0.97	55
def	0.94	0.98	0.96	123
date	1.00	1.00	1.00	47
reason	1.00	1.00	1.00	6
event	1.00	1.00	1.00	7
food	1.00	1.00	1.00	3
color	1.00	1.00	1.00	10
termeq	1.00	0.75	0.86	4

Table 10.4: Best performing fine-grained classes.

Class	Precision	Recall	F1-Score	Support	Issue
title	0.00	0.00	0.00	1	No correct predictions
product	0.50	0.50	0.50	2	Low support
period	0.50	0.67	0.57	6	Confusion with other NUM classes
city	0.67	1.00	0.80	2	Low precision
substance	0.60	1.00	0.75	3	Low precision
other (LOC)	1.00	0.25	0.40	4	Very low recall

Table 10.5: Worst performing fine-grained classes.

10.3.2 Confusion matrix (fine-grained)

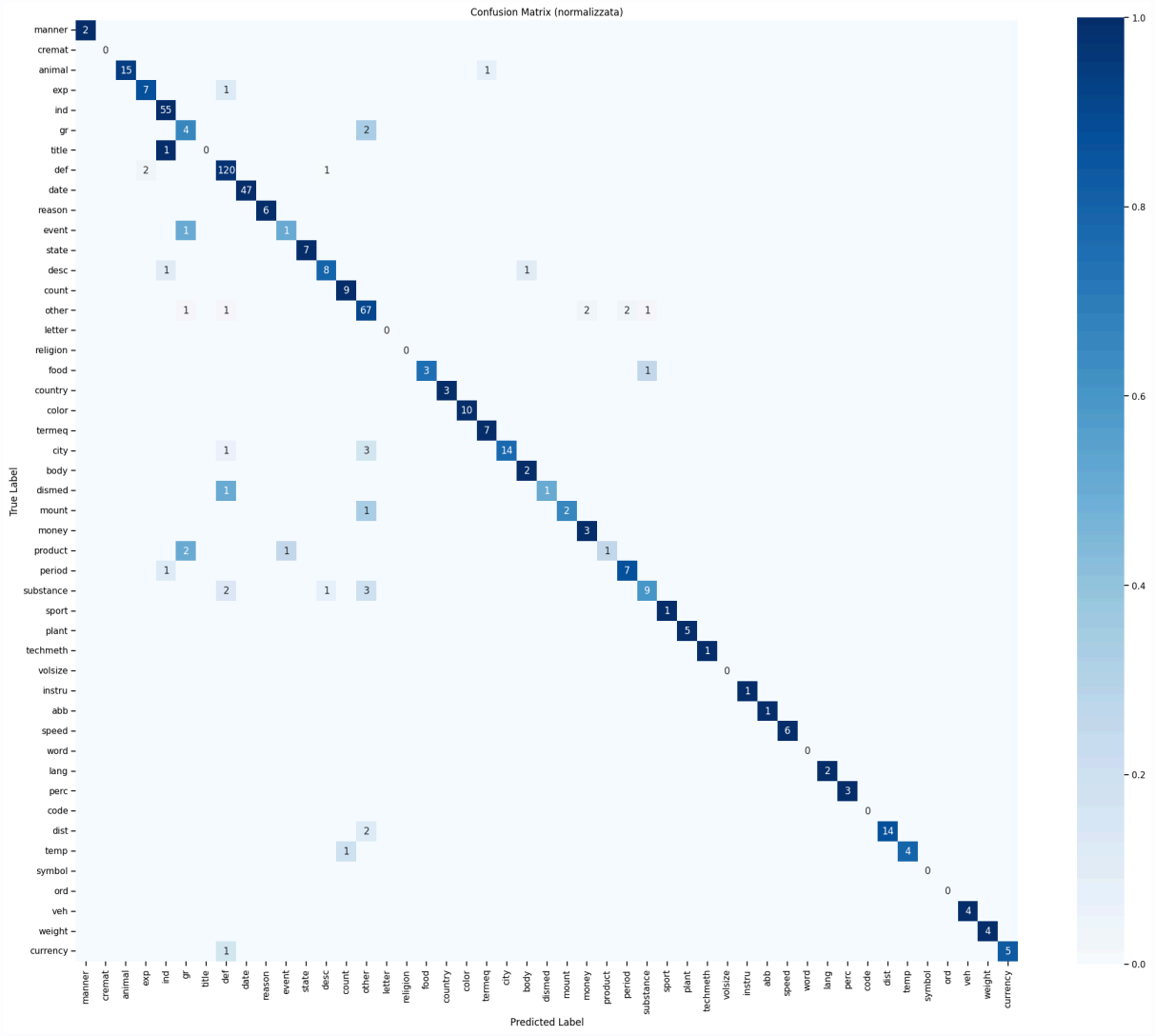

Figure 10.1: Normalized confusion matrix for fine-grained classification. Diagonal elements represent correct classifications; off-diagonal elements represent errors.

The confusion matrix reveals that most classes cluster strongly along the diagonal, indicating correct classification. However, several off-diagonal patterns emerge, which we analyze in detail below.

10.4 Most confused class pairs

True label	Predicted label	Count	Likely reason
substance	other (ENTY)	3	Semantic overlap in entity descriptions
city	other (ENTY)	3	Location entities confused with general entities
period	other (ENTY)	2	Temporal entities vs general entities
def	manner	2	Definitional vs procedural descriptions
other (ENTY)	other (LOC)	2	Ambiguous “other” categories
product	other (ENTY)	2	Product entities vs general entities

dist	other (ENTY)	2	Distance queries confused with entities
title	ind	1	Role vs individual person
def	desc	1	Definition vs general description

Table 10.6: Top 10 most confused fine-grained class pairs

Key error patterns:

1. **LOC subclasses → ENTY:other:** multiple location subclasses (city, state, country, mount) are frequently misclassified as ENTY:other. This explains the severe LOC recall problem observed at the coarse level.
2. **Semantic ambiguity in "other" classes:** the "other" categories in both ENTY and LOC serve as catch-all classes, leading to confusion when questions don't fit neatly into specific subcategories.
3. **NUM subclass confusion:** some numeric subclasses (period, dist) are confused with ENTY:other, suggesting difficulty distinguishing between questions asking "what" (entity) vs. "how much/many" (numeric).

10.5 Error analysis by question Word

Question Word	Accuracy	Correct	Total	Error Rate
When	100.0%	26	26	0.0%
Which	100.0%	6	6	0.0%
Why	100.0%	4	4	0.0%
Name	100.0%	2	2	0.0%
Other	100.0%	13	13	0.0%
Who	97.9%	46	47	2.1%
How	93.9%	31	33	6.1%
Where	92.3%	24	26	7.7%
What	90.1%	309	343	9.9%

Table 10.6: Classification accuracy by question word.

Key observations:

1. **Perfect accuracy for specific question words:** questions beginning with "When", "Which", "Why", and "Name" achieve 100% accuracy. These question words have strong, unambiguous associations with specific answer types (When→NUM:date, Why→DESC:reason).
2. **"Who" is near-perfect:** only 1 out of 47 "Who" questions is misclassified, confirming the strong HUM association.
3. **"What" most challenging:** questions beginning with "What" have the lowest accuracy (90.1%), accounting for 34 of the 39 total errors. This is expected given that "What" is the most ambiguous question word, distributed across all coarse categories.
4. **"Where" underperforms:** despite "Where" having a strong association with LOC (93.4% in training data), accuracy is only 92.3%. This is consistent with the LOC recall problem identified earlier.

10.6 Error distribution analysis

True class	Most common error	Error count	Class Total	Error Rate
LOC:other	ENTY:other	3	4	75.0%
ENTY:substance	ENTY:other	3	15	40.0%
ENTY:period	ENTY:other	2	6	33.3%
LOC:city	ENTY:other	3	18	22.2%
ENTY:product	ENTY:other	2	4	50.0%
NUM:dist	ENTY:other	2	16	12.5%

Table 10.7: Classes with highest error rates and their most common misclassification targets.

The pattern is clear, ENTY:other acts as an "attractor" class, absorbing questions from multiple categories (especially LOC subclasses and less common ENTY subclasses).

10.9 Root cause analysis

Based on the error analysis, we identify the following root causes:

Root cause	Evidence	Affected classes	Potential solution
LOC/ENTY semantic overlap	54 LOC → ENTY errors	LOC:city, LOC:state, LOC:other	Add location-specific features or use hierarchical classification
"Other" category ambiguity	Multiple classes → ENTY:other	ENTY:substance, ENTY:product, LOC:other	Better representation for catch-all categories
"What" question ambiguity	90.1% accuracy vs 97-100% for other words	All classes with "What" questions	Weight question words more heavily or use separate classifiers
Class imbalance	Rare classes have high error rates	HUM:title, ENTY:product	Data augmentation or class-weighted training
Fine-grained similarity	DESC:def ↔ DESC:desc confusion	DESC subclasses	May require deeper semantic understanding

Table 10.8: Root cause analysis of classification errors

The error analysis reveals that while the model achieves strong overall performance (92.2%), the errors are highly concentrated in specific patterns—particularly the confusion between LOC and ENTY categories. Addressing the LOC→ENTY confusion alone could potentially improve accuracy by approximately 5-10 percentage points.

11. Conclusions

This project successfully demonstrated that clustering-derived features can enhance question classification performance on the TREC dataset, even when clustering quality metrics suggest modest cluster separation.

Aspect	Key Finding
Best Configuration	TF-IDF + SBERT +Clusters + Linear SVM
Bast Accuracy	0.922

Best F1-Macro	0. 866
Best F1-Weighted	0. 919
Silhouette Score	0.0651

Our systematic experimentation yielded several important findings.

1. **Validation of the clustering-for-features approach**: adding cluster-based soft assignment features to TF-IDF representations improved accuracy by 5.0% (linear SVM) and 9.0% (logistic regression), confirming that unsupervised clustering captures useful semantic structure for supervised classification;
2. **Feature complementarity**: the combination of lexical (TF-IDF), semantic (SBERT), and structural (cluster) features achieved the best performance (92.0% accuracy, 86.4% F1-Macro), indicating that these representations capture different aspects of question semantics;
3. **Effectiveness of soft assignment features**: the 139-dimensional soft assignment feature vector (distances and similarities to 58 centroids) proved effective for capturing degrees of cluster membership and encoding semantic positioning relative to learned prototypes;
4. **Importance of classifier choice**: linear SVM consistently outperformed logistic regression by 10-17 percentage points, emphasizing that classifier choice is as important as feature engineering for this task.

Overall, this work illustrates the value of combining **classical NLP techniques (TF-IDF)**, **modern neural representations (SBERT)**, and **unsupervised learning (clustering)** within a principled and methodologically sound framework. The strong performance achieved by the full model demonstrates that thoughtful feature engineering remains a powerful approach, even in the presence of advanced pretrained embeddings.

Finally, the clustering-enhanced methodology presented in this project provides a **generalizable template** for other text classification tasks where capturing latent semantic structure may benefit supervised learning. Future work could explore alternative clustering algorithms, hierarchical classification strategies aligned with the TREC label structure, and more sophisticated feature fusion techniques to further improve performance.

12. References

[1] X. Li and D. Roth, "Learning Question Classifiers," in Proceedings of the 19th International Conference on Computational Linguistics (COLING 2002), Taipei, Taiwan, 2002, pp. 1-7. Available: https://aclanthology.org/C02-1150/

[2] TensorFlow Datasets, "TREC Question Classification Dataset," GitHub Repository. Available: https://github.com/tensorflow/datasets/tree/master/tensorflow_datasets/datasets/trec

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[4] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019.